

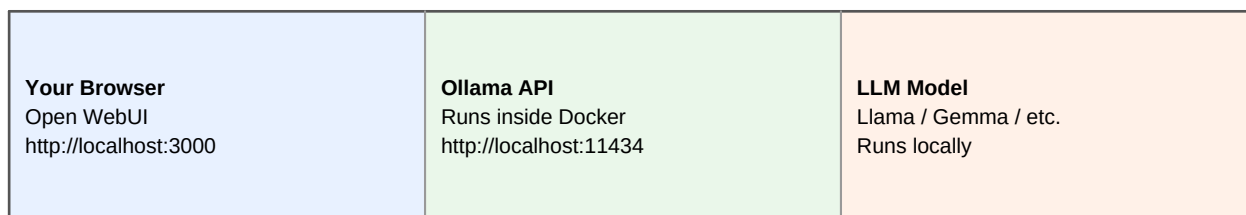
Beginner Guide: Run a Local LLM with Ollama, Docker & Open WebUI

A simple, practical introduction to running an AI model on your own computer.

Goal: Understand why local LLMs are useful, how Docker runs Ollama, and how Open WebUI gives you a browser-based chat interface.

1. Big Picture

Think of the setup as three layers:



Browser → Open WebUI → Ollama → LLM model → response comes back to your browser.

2. Why Run an LLM Locally?

- **Lower latency:** The request does not have to travel to a remote AI server. Local processing can reduce network-related delay.
- **Data privacy:** Your prompts and locally processed data can remain on your own machine, depending on your setup.
- **Security control:** You have more control over where the model and its data are running.
- **Cost control:** After the hardware is available, local inference can reduce per-request API charges.
- **Scalability:** You can increase local compute capacity when you need more processing power, although hardware limits still apply.
- **Reliability:** A local setup can continue working without an internet connection, provided the required model and software are already available.
- **Maintainability:** You control the software, model versions, storage and configuration, although maintaining them becomes your responsibility.

3. Important Trade-off: Local LLMs Need Powerful Hardware

LLMs can be large. Larger models generally require more RAM/VRAM, storage and processing power. Running everything directly on a computer can therefore be demanding.

The idea in this guide: use Docker to package and run the LLM service in an isolated container. Docker does not magically make an LLM lightweight; it makes deployment and management easier and more consistent.

4. What Are Docker, Image, Container and Volume?

Term	Simple meaning
Docker	A platform used to package and run applications in containers.
Image	A packaged template containing the software needed to create a container. Example: ollama/ollama.
Container	A running instance created from an image.

Term	Simple meaning
Volume	Persistent storage that keeps important data even when the container is recreated.
Port	A communication endpoint that lets applications outside the container reach a service inside it.

5. Step 1 — Install Docker Desktop

Install Docker Desktop on your computer and make sure Docker is running before executing the commands below. You can think of Docker Desktop as the tool that manages your Docker images, containers, volumes and networking.

6. Step 2 — Pull the Ollama Image

First download the Ollama Docker image from Docker Hub:

```
docker pull ollama/ollama
```

This downloads the image. It does **not** start Ollama yet.

7. Step 3 — Run the Ollama Container

```
docker run -d -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
```

Let's break this command into small pieces:

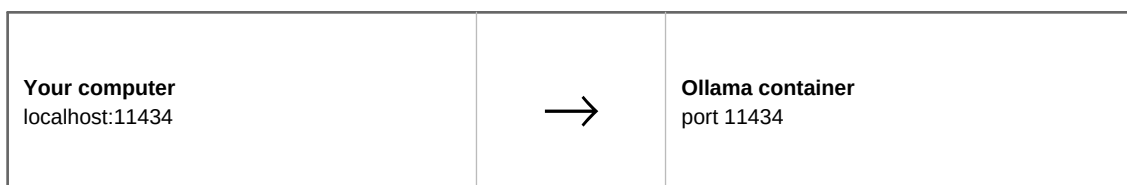
- **-d** — Run the container in the background.
- **-v ollama:/root/.ollama** — Mount the Docker volume named **ollama** into **/root/.ollama** inside the container. This provides persistent storage for Ollama data.
- **-p 11434:11434** — Map host port 11434 to container port 11434.
- **--name ollama** — Give the container the name **ollama**.
- **ollama/ollama** — The image from which the container is created.

8. What Does -p 11434:11434 Mean?

This is called **port mapping**. The format is:

```
HOST_PORT:CONTAINER_PORT
```

So **-p 11434:11434** means: send traffic arriving at port 11434 on your computer to port 11434 inside the Ollama container.



You can then test the Ollama service in a browser at **http://localhost:11434/**. A response from the service confirms that the endpoint is reachable.

9. Step 4 — Add a Web UI with Open WebUI

Ollama provides the model-serving layer. Open WebUI provides a convenient browser-based chat interface.

Important correction: the GitHub URL in the original notes points to WebLLM, which is a different project. For this Docker setup, use the official Open WebUI image.

```
docker pull ghcr.io/open-webui/open-webui:main
```

```
docker run -d -p 3000:3000 --name openwebui ghcr.io/open-webui/open-webui:main
```

Then open **http://localhost:3000/** in your browser.

Open WebUI documentation: <https://docs.openwebui.com/getting-started/quick-start/>

10. First Login

The notes provided a sample username/password of **admin@admin.com / admin**. Treat these as example credentials only. Open WebUI normally uses the account you create during its first-time setup; do not use a weak/default password for a real deployment.

11. Selecting and Using an LLM Model

Once Ollama is connected to Open WebUI, you can select an installed model from the model selector and send a prompt. The model generates the answer locally through Ollama.

Example model names from the original notes:

- ollama/llama3.1:8b
- ollama/llama3.1:13b
- ollama/llama3.1:70b
- ollama/llama3.2:8b
- ollama/llama3.2:13b
- ollama/llama3.2:70b

Reality check: availability and exact model tags can change. Also, a 70B model needs substantially more hardware than an 8B model.

12. Pulling Models from Open WebUI

You can manage models through the Open WebUI administration area when the Ollama connection is configured correctly.

- Open the **Admin** area.
- Go to **Models**.
- Use the **Pull** option to download a model.
- After the download finishes, the model becomes available for selection.

For example, you may pull a model such as **Gemma**, provided the model tag is supported by your configured Ollama version.

13. Essential Docker Commands

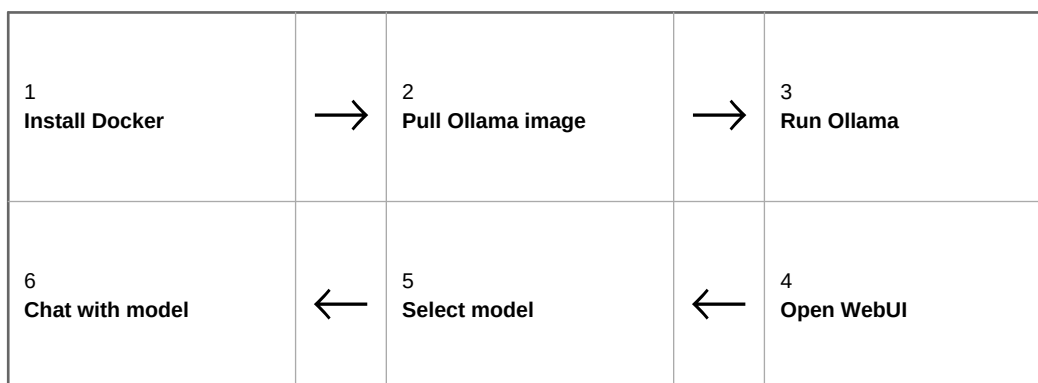
Command	Description
docker pull ollama/ollama	Pull the Ollama image.
docker run -d -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama	Create and run the Ollama container.

Command	Description
<code>docker ps</code>	List running containers.
<code>docker stop ollama</code>	Stop the Ollama container.
<code>docker start ollama</code>	Start a stopped Ollama container.
<code>docker restart ollama</code>	Restart the Ollama container.
<code>docker rm ollama</code>	Remove the Ollama container. Stop it first if it is running.
<code>docker rmi ollama/ollama</code>	Remove the Ollama image, if it is no longer in use.

14. Common Beginner Confusion

- **Image vs Container:** Image = blueprint/package. Container = running instance of that image.
- **Port vs Volume:** Port = network access. Volume = persistent storage.
- **Stopping vs Removing:** Stopping pauses a container. Removing deletes the container definition; the image and volume are separate resources.
- **Docker vs Ollama:** Docker runs the container. Ollama is the application inside the container that serves LLMs.
- **Open WebUI vs Ollama:** Open WebUI is the user interface. Ollama manages and serves the models.

15. Complete Setup Flow



16. Quick Checklist

- Docker Desktop is installed and running.
- Ollama image has been pulled.
- Ollama container is running.
- Port 11434 is mapped correctly.
- Ollama volume is mounted.
- Open WebUI image has been pulled.
- Open WebUI container is running on port 3000.
- Open WebUI can connect to Ollama.
- At least one LLM model is downloaded.
- The selected model can generate a response.

17. Key Takeaway

Docker provides the container environment → **Ollama** serves the LLM → **Open WebUI** provides the browser interface → **You** interact with the model through localhost.

This setup gives you a beginner-friendly path to experimenting with local AI. Remember that local deployment trades recurring cloud/API costs and network dependence for hardware requirements, local maintenance, storage and model-management responsibilities.